SALESFORCE CERTIFIED PLATFORM DEVELOPER

# Revolutionizing Agriculture with AgriEdge Or-Mange Ltd

*A Salesforce-Driven Order Management Solution*

*From manual order handling to an automated, secure and insight-driven agricultural supply chain.*

| | |
|---|---|
| **Topic** | Order Management System for AgriEdge Or-Mange Ltd |
| **Submitted by** | Karthikeyan M |
| **Institution** | Kongu Engineering College, Perundurai (B.E. Mechatronics Engineering) |
| **Programme** | Salesforce Certified Platform Developer, Skill Wallet (SmartBridge) |
| **Project mentor** | Rakesh Bhoomani |
| **Platform** | Salesforce Lightning, Developer Edition |

# Contents

1. Project Overview
2. Objectives
3. Phase 1: Requirement Analysis and Planning
4. Phase 2: Salesforce Development, Backend and Configuration
5. Phase 3: Application Interface and User Management
6. Phase 4: Data Migration, Testing and Security
7. Phase 5: Deployment, Documentation and Maintenance
8. Conclusion
9. Future Enhancements

## Summary of results

| Area | Result |
|---|---|
| Custom objects | 4 (Order, Order Item, Inventory, Shipment) |
| Validation rules | 2 (shipment tracking rule, inventory reorder alert) |
| Automation | 1 record-triggered Flow calling an Apex action |
| Apex | 6 classes, 3 triggers, 1 test class (8 test methods) |
| Test result | All tests passed, 88% overall code coverage |
| Security | 4 roles, 3 custom profiles, 3 users, field-level security on 3 fields |

# 1. Project Overview

AgriEdge Or-Mange Ltd is an agriculture and food production company that supplies seeds, fertilizers, harvested crops and processed food items. Its order handling depended on manual steps, which led to data-entry errors, delayed order updates, poor visibility of stock levels and disconnected customer service. This project delivers a Salesforce-driven Order Management System (OMS) that brings customers, orders, order items, inventory and shipments into one secure platform, with automation that removes repetitive work and business rules that protect data quality.

***Key features***

- **Connected data model:** four custom objects (Order, Order Item, Inventory and Shipment) linked to the standard Account and Product objects.
- **Automatic calculations:** auto-numbered orders (ORD-0001 format), line totals for every order item and a 10% discounted total on each order.
- **Stock awareness:** a Stock Status formula that flags products at or below their reorder level, plus a validation alert for low stock.
- **Automation:** a record-triggered Flow that creates follow-up tasks for new orders, and Apex triggers that update order status, order totals, shipments and customer emails.
- **Layered security:** a role hierarchy, three custom profiles and field-level security that hides financial fields from selected users.
- **Quality assurance:** an Apex test class that passes completely with 88% code coverage.

# 2. Objectives

The main goal of this project is to replace manual order handling at AgriEdge Or-Mange Ltd with one automated, secure and well-organised CRM. The system gives the sales, warehouse and finance teams the information they need, when they need it, so that orders are processed faster, stock is managed more accurately and customers receive reliable service. Each objective below is linked to a clear business benefit.

| Objective | Business value |
|---|---|
| Automate order processing | Fewer manual errors, faster handling and consistent order status updates. |
| Track inventory in real time | Prevents stock-outs and overstock by flagging products that reach their reorder level. |
| Connect orders, items and shipments | One complete view of every customer order from creation to delivery. |
| Protect sensitive data | Only authorised roles and profiles can see financial fields and perform restricted actions. |
| Enforce business rules | No order can be delivered without a tracking number, and low stock is flagged immediately. |

| Objective | Business value |
|---|---|
| Support decision making | Salesforce reports and dashboards can summarise order trends, inventory levels and shipment performance. |

# 3. Phase 1: Requirement Analysis and Planning

## 3.1 Understanding Business Requirements

The business analysis started from the use case provided for AgriEdge Or-Mange Ltd. The company works with a wide range of products and a complex supply chain, and it faced five recurring problems. The table maps each problem to the part of the OMS that addresses it.

| Business problem | Impact | How the OMS solves it |
|---|---|---|
| Manual order processing errors | Wrong totals and delayed orders | Formula fields, validation rules and automatic status updates through Apex. |
| No real-time inventory visibility | Stock-outs and overstock | Inventory object with a Stock Status formula and a low-stock validation alert. |
| Disconnected customer service | Slow answers to customer queries | Orders linked to the customer Account, related items and shipments on one record, and an email when payment is received. |
| Weak protection of business data | Risk of exposing prices and payments | Role hierarchy, custom profiles and field-level security. |
| Limited reporting | Decisions taken without data | Reports and Dashboards tabs in the AgriEdge app, ready for order, stock and shipment reporting. |

## 3.2 Project Scope and Objectives

***In scope***

- Order management with order items, payment status and discounted totals.
- Inventory management with reorder levels and stock status.
- Shipment tracking with carrier and shipment status.
- Automation using Flow and Apex, with test classes.
- Role-based access control, profiles, users and field-level security.
- Lightning app, tabs and page layouts for end users.

***Out of scope for this release***

- External payment gateway and courier integrations.
- Customer self-service portal (Experience Cloud) and mobile app.
- Bulk data migration from legacy systems.

## 3.3 Data Model Design

The data model has four custom objects that extend the standard Account and Product objects. An order belongs to a customer Account and holds many order items. Each order item points to

a Product. Shipments belong to an order, and inventory records belong to a Product. The diagram shows these relationships.

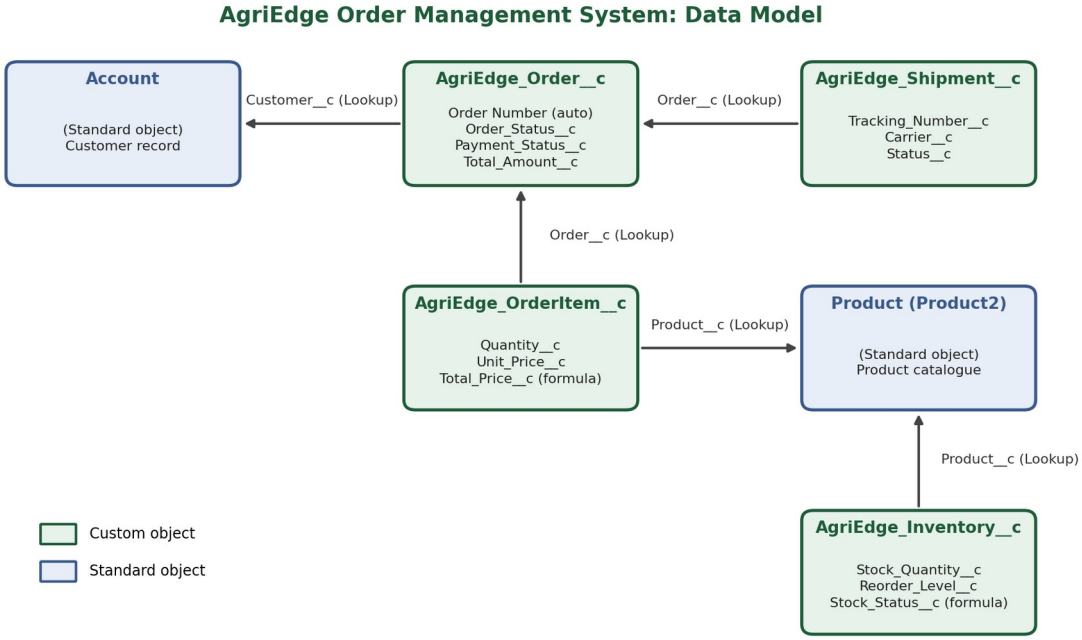
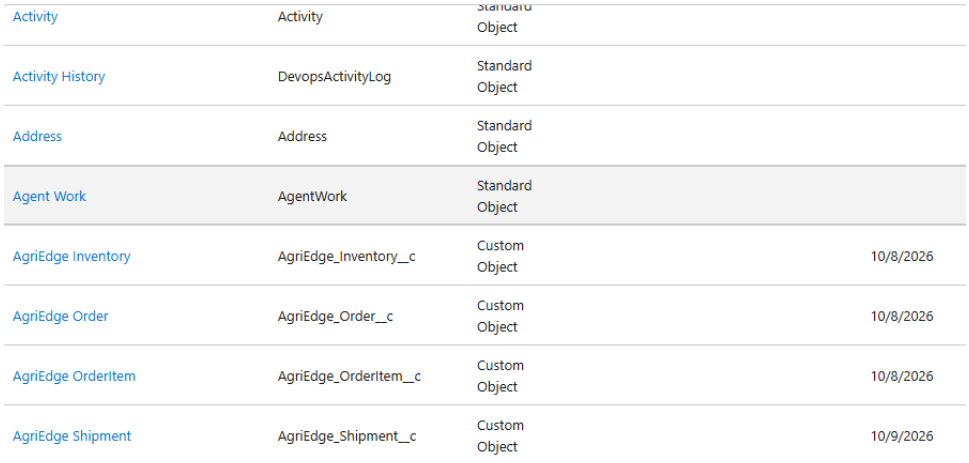



***Figure 1:*** *Data model of the AgriEdge Order Management System*

| Object | API name | Purpose | Related to |
|---|---|---|---|
| AgriEdge Order | AgriEdge_Order__c | Customer order with status, payment and totals | Account |
| AgriEdge OrderItem | AgriEdge_OrderItem__c | One product line within an order | AgriEdge Order, Product |
| AgriEdge Inventory | AgriEdge_Inventory__c | Stock quantity and reorder level for a product | Product |
| AgriEdge Shipment | AgriEdge_Shipment__c | Delivery details of an order | AgriEdge Order |



| | | | |
|---|---|---|---|
| Activity | Activity | Standard Object | |
| Activity History | DevopsActivityLog | Standard Object | |
| Address | Address | Standard Object | |
| Agent Work | AgentWork | Standard Object | |
| AgriEdge Inventory | AgriEdge_Inventory__c | Custom Object | 10/8/2026 |
| AgriEdge Order | AgriEdge_Order__c | Custom Object | 10/8/2026 |
| AgriEdge OrderItem | AgriEdge_OrderItem__c | Custom Object | 10/8/2026 |
| AgriEdge Shipment | AgriEdge_Shipment__c | Custom Object | 10/9/2026 |

***Figure 2:*** *The four custom objects in Object Manager*

### 3.4 Security Model Design

Security is designed in three layers. Roles control which records users can see through the hierarchy. Profiles control what users can do with each object. Field-level security controls which individual fields users can see. Users are assigned a role and a profile that match their job.

| Role | Reports to | Typical responsibility |
|---|---|---|
| CEO | Top of hierarchy | Overall visibility of all records |
| Sales Representative | CEO | Creates and follows up customer orders |
| Finance Team | Sales Representative | Reviews payments and totals |
| Warehouse Manager | CEO | Manages inventory and shipments |

## 4. Phase 2: Salesforce Development, Backend and Configuration

### 4.1 Environment Setup and DevOps Workflow

The solution was built in a free Salesforce Developer Edition org, which gives full access to the platform without affecting any production system. Configuration was done in Setup, Apex code in the Developer Console and automation in Flow Builder. Apex source files are kept in a GitHub repository for version control, and the test class is run after every code change so that problems are found early.

| Tool | Use in this project |
|---|---|
| Salesforce Developer Edition org | Build, test and demonstrate the whole solution |
| Setup and Object Manager | Objects, fields, validation rules, tabs, profiles and roles |
| Flow Builder | Record-triggered automation |
| Developer Console | Apex classes, triggers, tests and code coverage |
| GitHub | Version control for Apex source code and documentation |

### 4.2 Objects, Fields and Formula Fields

All custom fields follow the Salesforce naming convention, so Order Status becomes Order_Status__c. The tables list the fields created on each object.

***AgriEdge Order***

| Field label | API name | Type | Details |
|---|---|---|---|
| Order Number | Name | Auto Number | Format ORD-{0000} |
| Customer | Customer__c | Lookup | Account |
| Order Status | Order_Status__c | Picklist | New, Processing, Shipped, Delivered, Canceled |
| Order Date | Order_Date__c | Date/Time | Date the order was placed |
| Total Amount | Total_Amount__c | Currency (16, 2) | Sum of order item totals |

| Field label | API name | Type | Details |
|---|---|---|---|
| Payment Status | Payment_Status__c | Picklist | Pending, Paid, Failed |
| Shipping Address | Shipping_Address__c | Text Area (255) | Delivery address |
| Discounted Total | Discounted_Total__c | Formula (Currency) | Total_Amount__c - (Total_Amount__c * 0.1) |

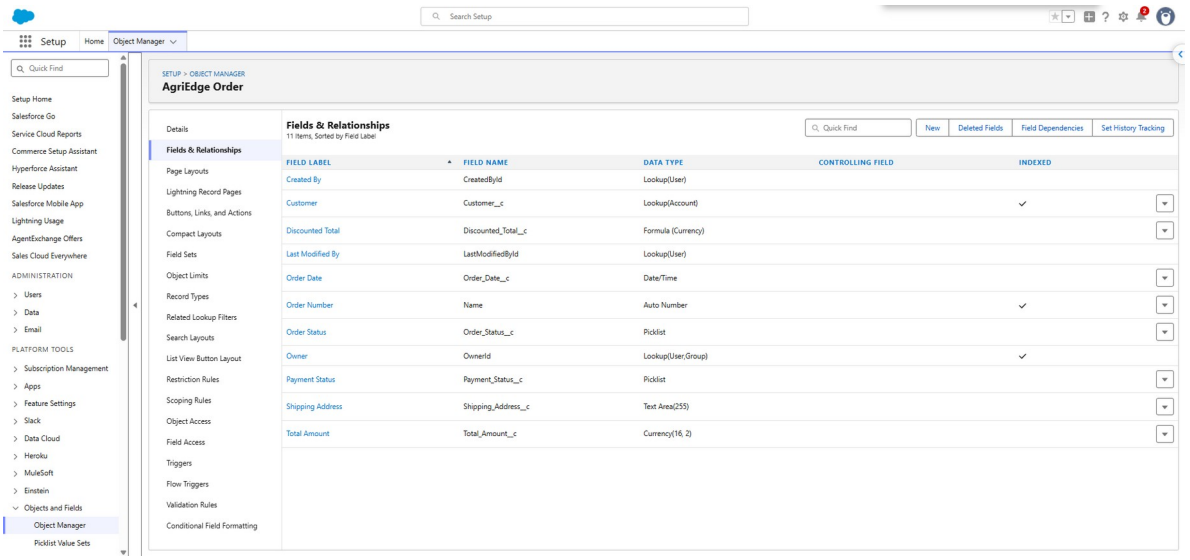



***Figure 3:*** *Fields and relationships of the AgriEdge Order object*

### *AgriEdge OrderItem*

| Field label | API name | Type | Details |
|---|---|---|---|
| Order | Order__c | Lookup | AgriEdge Order |
| Product | Product__c | Lookup | Product |
| Quantity | Quantity__c | Number (16, 0) | Units ordered |
| Unit Price | Unit_Price__c | Currency (16, 2) | Price per unit |
| Total Price | Total_Price__c | Formula (Currency) | Quantity__c * Unit_Price__c |

### *AgriEdge Inventory*

| Field label | API name | Type | Details |
|---|---|---|---|
| Product | Product__c | Lookup | Product |
| Stock Quantity | Stock_Quantity__c | Number (16, 0) | Units in stock |
| Reorder Level | Reorder_Level__c | Number (16, 0) | Level at which to restock |
| Warehouse Location | Warehouse_Location__c | Text (255) | Storage location |
| Stock Status | Stock_Status__c | Formula (Text) | IF(Stock_Quantity__c <= Reorder_Level__c, "Low", "Sufficient") |

### *AgriEdge Shipment*

| Field label | API name | Type | Details |
|---|---|---|---|
| Order | Order__c | Lookup | AgriEdge Order |
| Tracking Number | Tracking_Number__c | Text (100) | Courier tracking reference |
| Carrier | Carrier__c | Picklist | FedEx, UPS, DHL, Local Courier |
| Status | Status__c | Picklist | Pending, In Transit, Delivered |

## 4.3 Validation Rules

Two validation rules protect data quality. The shipment rule stops an order from being treated as delivered without a tracking number. The inventory rule warns users when stock reaches the reorder level.

| Rule | Object | Error condition | Error message and location |
|---|---|---|---|
| Order_Status_Change_Restriction | AgriEdge Shipment | AND(ISPICKVAL(Order__r.Order_Status__c, 'Delivered'), ISBLANK(Tracking_Number__c)) | "Tracking Number is required before marking order as Delivered." Shown on the Tracking Number field. |
| Inventory_Record_Alert | AgriEdge Inventory | Stock_Quantity__c <= Reorder_Level__c | "Stock is below reorder level. Please restock." Shown at the top of the page. |

## 4.4 Process Automation (Flow)

Salesforce has retired Process Builder, so the order-creation automation was built as a record-triggered Flow, which is the current recommended tool. The Flow named Order Task Creator runs after an AgriEdge Order record is created with Order Status equal to New. It calls an Apex action that creates a high-priority task for the Platform 1 users, asking them to create the order item records.

| Setting | Value |
|---|---|
| Flow name | Order Task Creator (record-triggered Flow) |
| Object and trigger | AgriEdge Order, when a record is created |
| Entry condition | Order Status equals New |
| Optimized for | Actions and Related Records (runs after the record is saved) |
| Action | Apex action createTaskForNewOrder from class OrderTaskCreator, with the order Record ID as input |

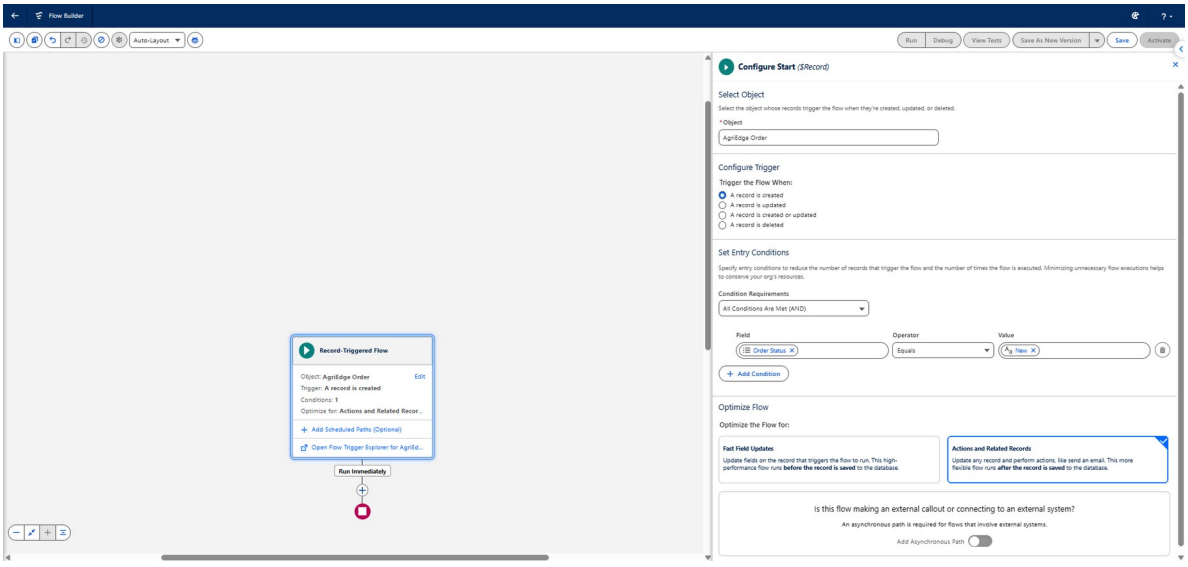



***Figure 4:*** *Flow start configuration: object, trigger and entry condition*

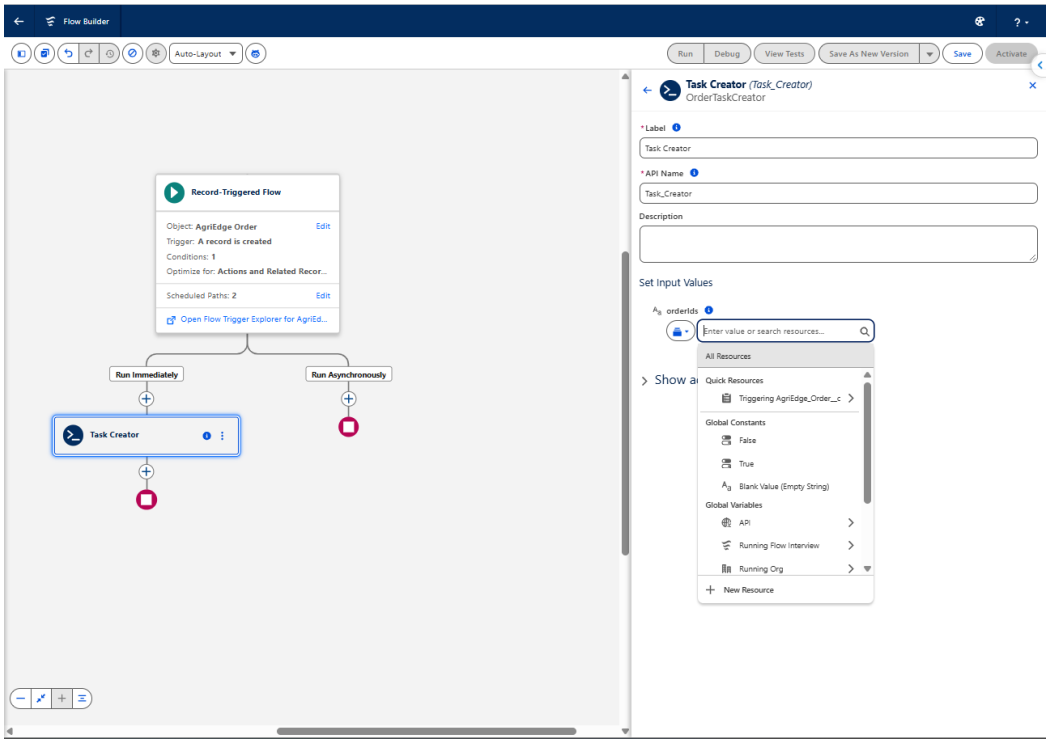



***Figure 5:*** *Flow canvas with the Apex action Task Creator*

## 4.5 Apex Classes and Triggers

Apex handles the logic that is too detailed for point-and-click tools. Each trigger stays short and hands its work to a helper class, which keeps the code easy to read, test and maintain.

| Component | Type | Purpose |
|---|---|---|
| OrderTaskCreator | Apex class (invocable) | Creates a "New Order Created" task for each Platform 1 user. Called from the Flow. |
| OrderStatusUpdater | Apex class | Moves orders that are still New to Processing. |

| Component | Type | Purpose |
|---|---|---|
| OrderTotalUpdater | Apex class | Adds up the order item totals into Total Amount and sets Payment Status to Pending when the total is above zero. |
| OrderItemTrigger | Trigger on AgriEdge OrderItem | After insert or update, calls OrderStatusUpdater and OrderTotalUpdater for the related orders. |
| OrderEmailSender | Apex class | Emails the customer contacts with the order summary when payment is received. |
| OrderPaymentStatusTriggers | Trigger on AgriEdge Order | Detects Payment Status changing to Paid and calls OrderEmailSender. |
| AgriEdgeOrderShipmentHelper | Apex class | Creates and updates shipments from order status, and cancels related items and shipments when payment fails. |
| AgriEdgeOrderTriggerHelper | Apex class | Holds a flag that prevents the order trigger from running recursively. |
| AgriEdgeOrderTrigger | Trigger on AgriEdge Order | Applies status rules for Pending and Failed payments and calls the shipment helper. |

### 4.6 Asynchronous Apex

No asynchronous Apex (Batch, Queueable or Schedulable classes) is part of this release. A scheduled batch job that checks all products against their reorder levels and alerts the warehouse team is listed as a planned enhancement in Section 9.

## 5. Phase 3: Application Interface and User Management

### 5.1 Lightning App and Tabs

A custom Lightning app named AgriEdge was created in App Manager so that users work from one focused workspace. Custom tabs were created for all four custom objects and set to Default On. The app navigation contains AgriEdge OrderItems, AgriEdge Inventories, AgriEdge Orders, AgriEdge Shipments, Reports and Dashboards, and the app is available to the System Administrator profile and the three Platform profiles.

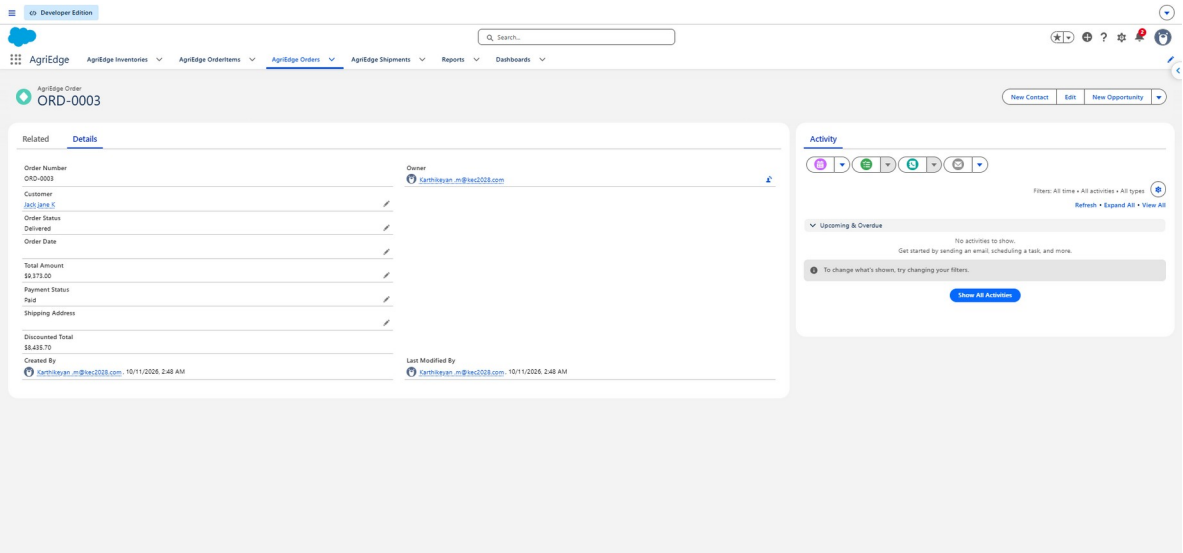



***Figure 6:*** *AgriEdge Order record opened in the AgriEdge Lightning app*

### 5.2 User Management

Three users were created to represent the main teams. Each user has the Salesforce Platform license, a role in the hierarchy and a matching profile.

| User | Role | Profile | License |
|---|---|---|---|
| John Production Engineer Sandbox 1 | Sales Representative | Platform 1 | Salesforce Platform |
| Quality Inspector Mike | Warehouse Manager | Platform 2 | Salesforce Platform |
| Plant Manager Albert | Finance Team | Platform 3 | Salesforce Platform |

## 6. Phase 4: Data Migration, Testing and Security

### 6.1 Data Loading

Test data for accounts, orders, order items, inventory and shipments was created through the Lightning user interface so that every feature could be tested with realistic records. For larger volumes, such as the first load of products and customers, the Data Import Wizard or Data Loader would be used with CSV files.

### 6.2 Field History, Duplicate and Matching Rules

Field history tracking was switched on at object level for the Order, Inventory and Shipment objects, so changes to tracked fields are recorded with the user and time. Duplicate rules, matching rules, permission sets and sharing rules were not needed for this release, because access is governed by roles, profiles and field-level security. They are listed as future enhancements.

## 6.3 Profiles, Roles and Field-Level Security

Three custom profiles were cloned from the Standard Platform User profile and given only the object permissions each team needs. The principle of least privilege was followed, so no profile can delete records.

| Profile | AgriEdge Order | AgriEdge OrderItem | AgriEdge Inventory | AgriEdge Shipment | Account |
|---|---|---|---|---|---|
| Platform 1 | Read, Create | Read, Create | No access | No access | Standard |
| Platform 2 | Read only | Read only | Read, Create, Edit | Read, Create, Edit | Read, Create, Edit |
| Platform 3 | Read, Create, Edit | Read, Create, Edit | No access | No access | Standard |

### *Field-level security*

The fields Payment Status, Total Amount and Discounted Total on the Order object were hidden from the Platform 1 and Platform 2 profiles. This was tested by logging in as the Sales Representative user, John, using the administrator login-access setting. The order record shows no payment or amount fields for him, while administrators still see them.

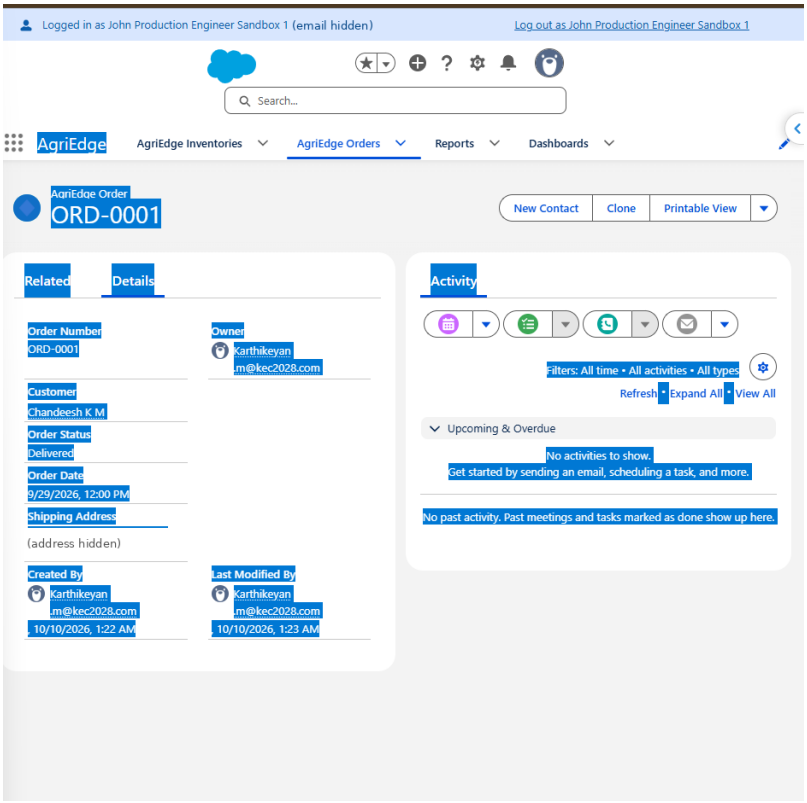



***Figure 7:*** *Order record as seen by a Platform 1 user: financial fields are hidden*

## 6.4 Test Classes

One test class, AgriEdgeOrderTests, tests all the Apex code. It contains eight test methods that create their own data, so they do not depend on records in the org. All tests passed and overall code coverage is 88%, which is above the 75% required for deployment.

| Test method | What it checks |
|---|---|
| testOrderTaskCreator | A task is created for each Platform 1 user and linked to the order. |
| testOrderStatusUpdater | Orders in New status move to Processing; other statuses are unchanged. |
| testOrderTotalUpdater | Order totals add up the item totals and Payment Status is set correctly. |
| testSendOrderEmail | The email method runs for orders with a customer and contacts. |
| testAgriEdgeOrderShipmentHelper | A shipment is created in Pending status for a processing order. |
| testAgriEdgeOrderTrigger | Changing Payment Status to Paid sets the order to Delivered. |
| testAgriEdgeOrderTriggerHelper | The recursion flag can be switched on and off. |
| testPaymentPaidSendsEmail | Changing Payment Status to Paid fires the payment trigger and email. |

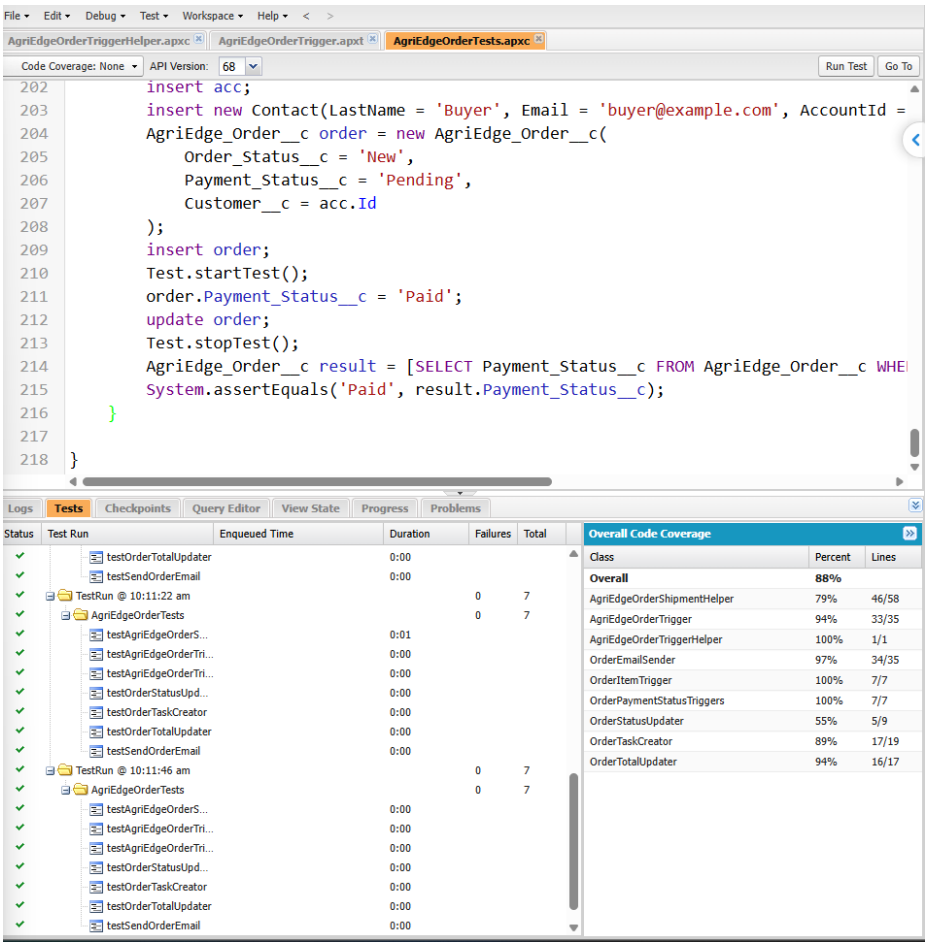



***Figure 8:*** *Test results and overall code coverage of 88%*

## 6.5 Test Cases

Each feature was tested with a clear input and an expected result. All test cases passed. Screenshots of the key results (field-level security, flow automation and Apex tests) are shown in this report.

| ID | Feature | Input | Expected result | Result |
|---|---|---|---|---|
| TC1 | Auto numbering | Create a new order | Order Number follows ORD-0001 format | Pass |
| TC2 | Discount formula | Total Amount $8,737.00 | Discounted Total shows $7,863.30 | Pass |
| TC3 | Shipment validation | Save a shipment for a Delivered order with no tracking number | Error: Tracking Number is required before marking order as Delivered. | Pass |
| TC4 | Shipment validation | Save the same shipment with a tracking number | Record saves successfully | Pass |
| TC5 | Inventory validation | Stock Quantity 5, Reorder Level 10 | Error: Stock is below reorder level. Please restock. | Pass |
| TC6 | Field-level security | Log in as the Platform 1 user and open an order | Payment Status, Total Amount and Discounted Total are hidden | Pass |
| TC7 | Flow automation | Create an order with Order Status New | Task "New Order Created" is assigned to the Platform 1 user | Pass |
| TC8 | Apex tests | Run AgriEdgeOrderTests | All tests pass with at least 75% coverage | Pass (88%) |

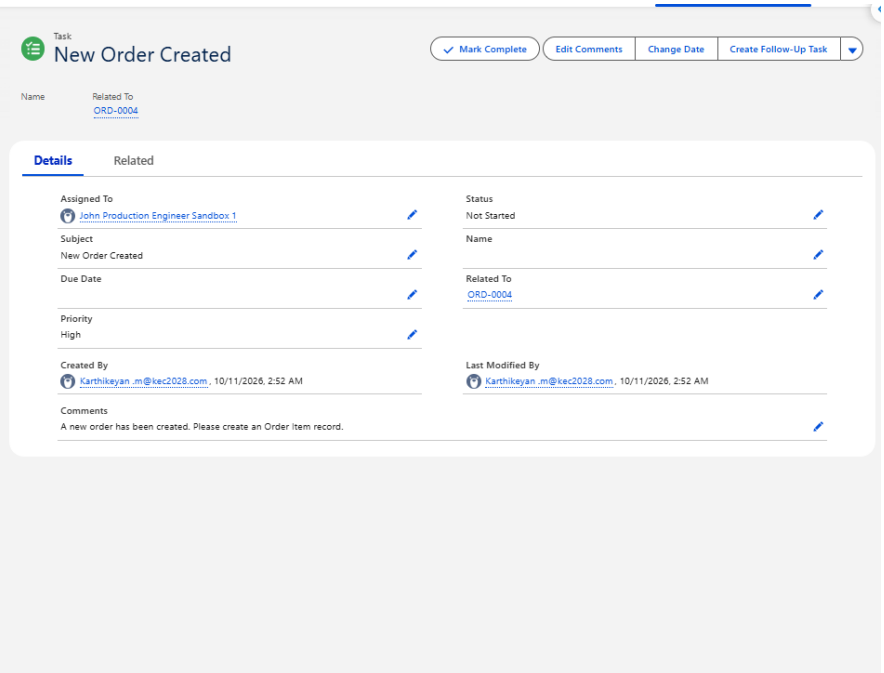



***Figure 9:*** *TC7: task created automatically for the new order*

# 7. Phase 5: Deployment, Documentation and Maintenance

## 7.1 Deployment Strategy

The recommended path from the Developer Edition org to a sandbox or production org is an outbound change set. The change set contains the four custom objects with their fields, validation rules, tabs, the AgriEdge app, profiles, the Flow and all Apex classes and triggers.

The deployment is validated first, the required Apex tests are run in the target org, and the deployment is completed only when all tests pass. For larger teams, the same components can be deployed with Salesforce CLI and version control.

### 7.2 Maintenance and Monitoring

- **Run tests regularly:** run AgriEdgeOrderTests after every change to keep coverage above 75%.
- **Watch for errors:** review failed Flow interviews, Apex jobs and debug logs in Setup.
- **Review access:** check profiles, roles and active users every quarter using the Setup Audit Trail.
- **Change safely:** make changes in a sandbox first and deploy them with a change set.
- **Protect data:** use the weekly data export to keep a backup of orders and inventory.

### 7.3 Troubleshooting Approach

Problems were solved by reading the exact error message, finding the component it points to, fixing that one component and testing again. The table lists the main issues met during development and how each was resolved.

| Issue | Cause | Resolution |
|---|---|---|
| "Variable does not exist" in the test class | A class the test calls had not been saved yet | Create and save classes in dependency order, then save the test class. |
| "No such column" for the order lookup | The supplied code used a different relationship field name than the data model | Updated the code to use the Order__c lookup field on Order Item and Shipment. |
| Field missing on the New record form | Page and field settings were cached in the browser | Hard refresh or a private window, then checked field-level security and the page layout. |
| Code coverage showed 0% after saving | Saving Apex clears stored coverage | Ran all tests again and waited for the run to finish. |
| Flow did not create a task | The test order was not created with status New | Created a new order with Order Status New, which fired the Flow. |
| Extra duplicate classes and triggers | Earlier attempts left copies | Deleted the duplicates so only the intended components run. |

## 8. Conclusion

The AgriEdge Order Management System shows how Salesforce can turn a manual, error-prone order process into an automated and secure one. The data model connects customers, orders, items, inventory and shipments. Validation rules and formulas protect data quality, a Flow and Apex code automate routine work, and roles, profiles and field-level security keep sensitive information with the right people. All Apex code was tested, all tests passed and coverage reached 88%. Every feature in this document was tested with defined inputs and expected results, and the evidence is included as screenshots. The solution is ready to be extended with reports, dashboards and integrations as AgriEdge Or-Mange Ltd grows.

## 9. Future Enhancements

- **Scheduled batch job:** check all products against reorder levels every day and alert the warehouse team.
- **Customer portal:** an Experience Cloud site where customers place orders and track shipments.
- **Payment and courier integration:** connect a payment gateway and courier tracking services.
- **AI and chatbot support:** Einstein predictions for demand forecasting and a chatbot for order enquiries.
- **Lightning Web Component:** a single-screen order entry component with live stock checks.
- **Advanced security:** permission sets, sharing rules, and duplicate and matching rules for accounts.
- **Notifications:** Slack or email alerts for low stock, failed payments and delayed shipments.